

```

forumbd=> select pk,title,category from delete_posts ;
pk | title | category
----+-----+-----
(0 rows)

```

2. Now suppose that we want to delete some records from the posts table, and we want all the records that we have deleted from the t_posts table to be inserted into the delete_posts table. To reach this goal, we have to use CTEs as follows:

```

forumbd=> with del_posts as (
    delete from t_posts
    where category in (select pk from categories where title
='Database Discussions')
    returning *)
insert into delete_posts select * from del_posts;
INSERT 0 2

```

The query here deletes all the records from the t_posts table that have their category as 'Database' and, in the same transaction, inserts all the records deleted in the delete_posts table, as we can see here:

```

forumbd=> select pk,title,category from t_posts ;
pk |          title          | category
----+-----+-----
  3 | A view of  Data types in C++ |        3
(1 row)

forumbd=> select pk,title,category from delete_posts ;
pk |          title          | category
----+-----+-----
  1 | Indexing PostgreSQL |        1
  2 | Indexing Mysql      |        1
(2 rows)

```

3. Now let's try another example by returning to the starting scenario:

```

forumbd=> drop table if exists t_posts;
DROP TABLE

```

```
forumbd=> create temp table t_posts as select * from posts;
SELECT 3
```

4. As we have done before, let's create a new table named `inserted_post` with the same data structure as the `posts` table:

```
forumbd=> create table inserted_posts as select * from posts limit
0;
SELECT 0
```

5. Suppose now that we want to perform a SQL query that moves, in the same transaction, all the records that are present in the `t_posts` table to the `inserted_posts` table. This query will be as follows:

```
forumbd=> with ins_posts as ( insert into inserted_posts select *
from t_posts returning pk) delete from t_posts where pk in (select
pk from ins_posts);
DELETE 3
```

As we can see from the results, the query has achieved our goal:

```
forumbd=> select pk,title,category from t_posts ;
pk | title | category
----+-----+-----
(0 rows)

forumbd=> select pk,title,category from inserted_posts ;
pk | title | category
----+-----+-----
1 | Indexing PostgreSQL | 1
2 | Indexing Mysql | 1
3 | A view of Data types in C++ | 3
(3 rows)
```

Query recursion

In PostgreSQL, it is possible to create recursive queries. Recursive queries are used in graph databases and in many common use cases, such as querying tables that represent website menus. Recursive CTEs make it possible to have recursive queries in PostgreSQL.

Recursive CTEs

A recursive CTE is a special construct that allows an auxiliary statement to reference itself and, therefore, join itself onto previously computed results. This is particularly useful when we need to join a table an unknown number of times, typically to “explode” a flat tree structure. The traditional solution would involve some kind of iteration, probably by means of a cursor that iterates one tuple at a time over the whole resultset. However, with recursive CTEs, we can use a much cleaner and simpler approach. A recursive CTE is made by an auxiliary statement that is built on top of the following:

- A non-recursive statement, which works as a bootstrap statement and is executed when the auxiliary term is first evaluated
- A recursive statement, which can either reference the bootstrap statement or itself

These two parts are joined together by means of a UNION predicate. For example, let’s insert a new record in the tag table and then see inside:

```
forumdb=> insert into tags (tag,parent) values ('PostgreSQL',1);
INSERT 0 1

forumdb=> select * from tags order by pk;
pk |      tag      | parent
----+-----+-----
  1 | Database      |
  2 | Operating Systems |
  3 | PostgreSQL    |      1
(3 rows)
```

Now we would like to “explode” the flat tree structure and follow the relation between parent and child using the parent field of the tags table. So, we want the result to be something like this:

```
level |      tag
-----+-----
  1 | Database
  1 | Operating Systems
  2 | Database -> PostgreSQL
```

To reach this goal, we have to perform the following:

```
forumdb=> WITH RECURSIVE tags_tree AS (
  -- non recursive statement
```

```

SELECT tag, pk, 1 AS level
FROM tags WHERE parent IS NULL
UNION
-- recursive statement
SELECT tt.tag || ' -> ' || ct.tag, ct.pk
, tt.level + 1
FROM tags ct
JOIN tags_tree tt ON tt.pk = ct.parent
)
SELECT level,tag FROM tags_tree
order by level;
  level |          tag
-----+-----
      1 | Database
      1 | Operating Systems
      2 | Database -> PostgreSQL
(3 rows)

```



When we use CTEs, it is important to avoid infinite loops. These can happen if the recursion does not end properly.

Thus, we have learned how to use CTEs to tinker with tables.

Summary

Hopefully, this chapter was full of interesting ideas for the developer and the DBA. In this chapter, we talked about complex queries; we then saw the `SELECT` statement and the use of the `LIKE`, `ILIKE`, `DISTINCT`, `OFFSET`, `LIMIT`, `IN`, and `NOT IN` clauses. We then started talking about aggregates through the `GROUP BY` and `HAVING` clauses, and we introduced some aggregate functions, such as `SUM()`, `COUNT()`, `AVG()`, `MIN()`, and `MAX()`.

We then talked in depth about subqueries and joins. Another very interesting set of topics covered in this chapter was the `UNION`, `EXCEPT`, and `INTERSECT` queries. Finally, by looking at the advanced options for the `INSERT`, `DELETE`, `UPDATE`, and `MERGE` instructions, and by covering CTEs, we gave you an idea of the power of the SQL language owned by PostgreSQL.

As for the concept of aggregates, in the next chapter, we will see a new way to make aggregates using window functions. Through the use of window functions, we will see that we are able to create all the aggregates and aggregation functions described in this chapter, but we will also see that we have the option to create new ones.

Verify your knowledge

- If we run this query and data on the table called `mytable` is not changed, do we always get the same result?

```
select * from mytable
```

No, we don't, because the ordering of the data could be different.

See the section *Exploring the SELECT statement* for more details.

- Is it possible to have only 3 records as result of a query?

Yes, it's possible using the `LIMIT` clause.

See the section *Using limit and offset* for more details.

- If we have 2 tables: table A with 3 records with a field `id` as the primary key, and table B with 2 records with a field `id` as the primary key, what kind of join do we have to use to match all the records that have the same ID on table A and table B?

We have to use an inner join query:

```
select tableA.id from tableA inner join tableB using(id)
```

See the section *Using INNER JOIN* for more details.

- If we have 2 tables: table A with 3 records with a field `id` as the primary key, and table B with 2 records with a field `id` as the primary key, using the `NOT EXISTS` clause, how can we write a query that shows all the records that are in table A and not in table B?

```
select * from tableA where not exists (select 1 from tableB where  
tableA.id=tableB.id)
```

See the section *Subqueries and the EXISTS/NOT EXISTS condition* for more details.

- Do PostgreSQL 11 and PostgreSQL 16 have the same way of using CTEs?

No they don't. PostgreSQL 11 always materializes data. PostgreSQL 16, if not specified, materializes data only if the CTE is called twice or more inside the query.

See the section *Subqueries and the EXISTS/NOT EXISTS condition* and *Exploring CTE* for more details.

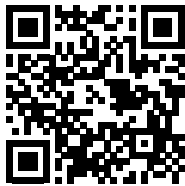
References

- Subquery expressions official documentation: <https://www.postgresql.org/docs/current/functions-subquery.html>
- Joins official documentation: <https://www.postgresql.org/docs/current/tutorial-join.html>
- CTEs official documentation: <https://www.postgresql.org/docs/current/queries-with.html>
- MERGE official documentation: <https://www.postgresql.org/docs/current/sql-merge.html>
- MERGE ANSI 2003 SQL: <https://www.w3resource.com/sql/sql-syntax.php>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



6

Window Functions

In the previous chapter, we talked about aggregates. In this chapter, we are going to further discuss another way to make aggregates: window functions. The official documentation (<https://www.postgresql.org/docs/current/tutorial-window.html>) describes window functions as follows:



A window function performs a calculation across a set of table rows that are somehow related to the current row. This is comparable to the type of calculation that can be done with an aggregate function. However, window functions do not cause rows to become grouped into a single output row as non-window aggregate calls would. Instead, the rows retain their separate identities. Behind the scenes, the window function is able to access more than just the current row of the query result

In this chapter, we will talk about window functions, what they are, and how we can use them to improve the performance of our queries.

The following topics will be covered in this chapter:

- Using basic statement window functions
- Using advanced statement window functions

Technical requirements

Before starting, remember to start the Docker container named `chapter_06` as shown below:

```
$ bash run-pg-docker.sh chapter_06
```


Using basic statement window functions

As we saw in the previous chapter, aggregation functions behave in the following way:

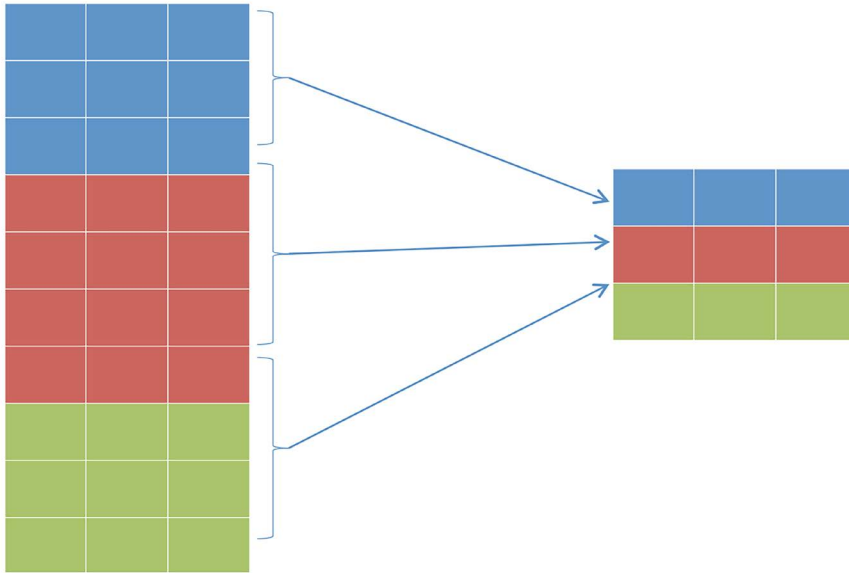


Figure 6.1: Standard group by aggregation

The data is first sorted and then aggregated; the data is then flattened through aggregation. This is what happens when we execute the following statement, after connecting as the forum user to forumdb database:

```
forumdb=> select category,count(*) from posts group by category order by
category;
category | count
-----+-----
1 | 2
3 | 1
(2 rows)
```

Alternatively, we can decide to use window functions by executing the following statement:

```
forumdb=> select category, count(*) over (partition by category) from
posts order by category;
category | count
-----+-----
```

```

1 | 2
1 | 2
3 | 1
(3 rows)

```

Window functions create aggregates without flattening the data into a single row. However, they replicate it for all the rows to which the grouping functions refer. The behavior of PostgreSQL is depicted in the following diagram:

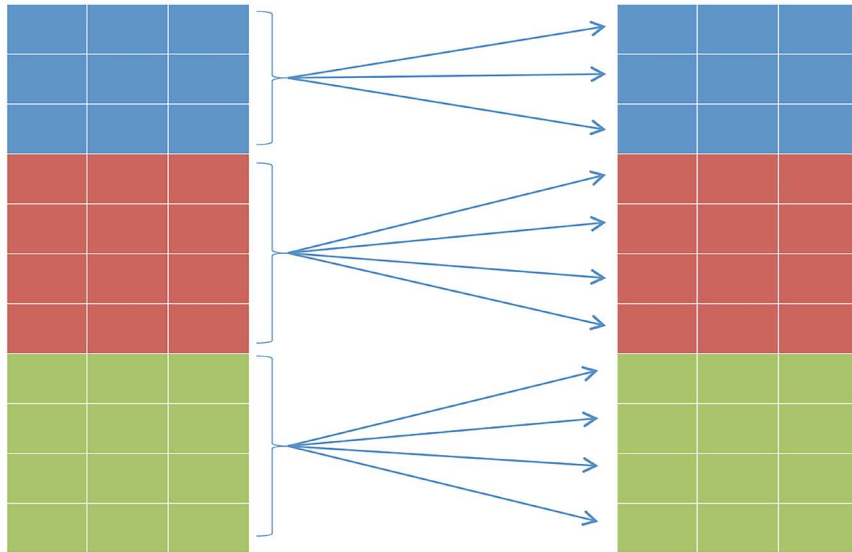


Figure 6.2: Window function aggregation

This is the reason that the `distinct` keyword has to be added to the preceding query if we want to obtain the same result that we get with a classic `GROUP BY` query.

Using the **PARTITION BY** function and **WINDOW** clause

Let's now run some basic queries using the window functions. Suppose that we want to use two `over` clauses. For example, if on one column we want to count the rows relating to the category, and on another column the total count of the columns, then we have to run the following statement:

```

forumbd=> select category, count(*) over (partition by category),count(*)
over () from posts order by category;
category | count | count
-----+-----+-----
1 | 2 | 3
2 | 2 | 3
3 | 1 | 3

```